An Experimental Analysis of Deep RL Architectures in Pac-Man

Mingxuan Zhang
May 2026

1 Introduction

1.1 Motivation: Why Architecture Matters in Extreme Game Environments

The choice of deep RL architecture—how data is collected, how credit is assigned, and how policies are updated—has outsized consequences in environments where the margin for error is zero. In standard benchmarks such as Atari or MuJoCo, the distinction between offline and online methods, or between value-based and policy-gradient approaches, often manifests as differences in sample efficiency or asymptotic performance. In *extreme* environments—those combining sparse rewards, non-stationary opponent dynamics, and zero tolerance for failure—the same architectural choices can determine whether learning succeeds at all.

This paper uses the classic arcade game Pac-Man as a controlled testbed to conduct a systematic, paradigm-comparative analysis of three architectures spanning the spectrum of contemporary deep RL. Our contribution is not a new algorithm, but a detailed **mechanistic diagnosis** of *why* two widely-used paradigms fail in this regime, and *which* structural properties of the third enable it to succeed. We then leverage this diagnosis to design a targeted observation-space intervention that addresses the residual failure mode of the best-performing architecture.

1.2 Pac-Man as a Diagnostic Testbed

Pac-Man occupies a unique position in the landscape of RL benchmarks. Unlike grid-world navigation or single-agent continuous control, it combines five stress dimensions into a single environment, summarized in Table 1.

Table 1: Stress-test dimensions of Pac-Man compared to standard RL benchmarks

Dimension	**Standard Benchmarks**	**Pac-Man**
Reward density	Dense / semi-dense	Extremely sparse (pellet +1, clear +50)
Fault tolerance	Multi-episode recovery	**Zero tolerance** (1 death = mission failure)
Opponent dynamics	Static or single-agent	**4 independent ghost AIs**, periodic mode switching
Kinematic stability	Stationary	**Non-stationary** (power pellet inverts predator–prey roles)
State observability	Fully observable	Ghost identity / direction / intent entirely implicit

These five dimensions are not merely additive—they interact. Sparse rewards amplify the consequences of distribution shift; non-stationary opponent dynamics make historical data stale;

zero fault tolerance converts any inference error into a terminal event. This interaction makes Pac-Man an ideal **diagnostic instrument** for exposing architectural failure modes that may remain latent in more forgiving benchmarks.

1.3 Environment Specification

We employ a custom-built 28×31 arcade-accurate Pac-Man simulator. The four ghosts (Blinky, Pinky, Inky, Clyde) each run an independent, deterministic AI: a periodic Scatter/Chase mode schedule ($[42, 120, 42, 120, 30, 120, 30, \infty]$), greedy Euclidean-distance routing toward a ghost-specific chase target, and a fixed tie-breaking priority (UP $>$ LEFT $>$ DOWN $>$ RIGHT). At difficulty level 2, the Cruise Elroy mechanism causes Blinky to ignore Scatter mode and pursue Pac-Man directly when remaining pellets drop to ≤ 20. The action space consists of four directions with a no-reverse constraint. An episode terminates upon pellet clearance, 3,000 elapsed steps, or loss of all lives.

1.4 Architectures Under Investigation

We evaluate three architectures spanning the methodological spectrum of deep RL:

1. **DecisionTransformer** (offline sequence modeling). A GPT-2-style causal Transformer conditioned on return-to-go, state, and action triplets. Pre-trained via behavioral cloning on expert trajectories, then fine-tuned online with TD-error. Represents the *offline-to-online transfer* paradigm.
2. **DAgger + DQN** (online imitation with value-based correction). A pre-trained PPO policy serves as teacher; online rollouts are iteratively aggregated with teacher annotations into a mixed dataset on which a DQN is fine-tuned. Represents the *imitation-learning with self-correction* paradigm.
3. **PPO + GAE** (on-policy self-play). Pure online Proximal Policy Optimization with Generalized Advantage Estimation, augmented with Random Network Distillation for intrinsic exploration and a curriculum over three difficulty levels. Represents the *on-policy self-play* paradigm.

All three architectures are evaluated on the identical Pac-Man engine, eliminating environment-level confounds and enabling direct attribution of performance differences to architectural properties.

1.5 Architectural Pseudocode

Algorithms 1–3 present the core training loops of the three architectures at a level of abstraction that exposes their structural differences—in particular, how each handles the data-collection–credit-assignment–policy-update pipeline.

The structural distinctions are immediately visible: DecisionTransformer relies on a pre-collected expert dataset and conditions on return-to-go (line 5–10), creating a dependency on the training distribution that persists through online fine-tuning (line 13–18). DAgger interleaves teacher annotation with student exploration (line 5–8), but the teacher's annotations enter the Bellman target y (line 11), coupling credit assignment to the teacher's expressive ceiling. PPO maintains a strictly co-evolving data stream (lines 4–7) with no external dataset or teacher, and GAE (lines 9–10) decomposes credit at the step level before the PPO clip objective (line 14) constrains the policy update magnitude.

Algorithm 1 DecisionTransformer: offline-to-online transfer

Input: Expert trajectories $\mathcal{D}_{\text{expert}}$, context length K
Stage 1: BC Pre-training
for each epoch **do**
  for each sequence $\tau = (R_1, s_1, a_1, \ldots, R_K, s_K, a_K) \sim \mathcal{D}_{\text{expert}}$ **do**
    $\hat{a}_t \leftarrow \text{DT}(R_{<t}, s_{<t}, a_{<t}, t)$ ▷ causal self-attention over context
    $\mathcal{L}_{\text{BC}} \leftarrow -\sum_t \log p(\hat{a}_t = a_t)$
    Update DT parameters via $\nabla \mathcal{L}_{\text{BC}}$
  end for
end for
Stage 2: Online Fine-tuning
for each episode **do**
  $R_0 \leftarrow$ target return, $s_0 \leftarrow$ env.reset()
  for $t = 0, 1, \ldots$ until done **do**
    $\hat{a}_t \leftarrow \text{DT}(R_{<t}, s_{<t}, a_{<t}, t)$
    $s_{t+1}, r_t \leftarrow \text{env.step}(\hat{a}_t)$
    $R_{t+1} \leftarrow R_t - r_t$ ▷ decrement return-to-go
    Store (s_t, a_t, r_t, s_{t+1}) in replay buffer $\mathcal{B}$
    Sample batch $\sim \mathcal{B}$, compute TD-error, update DT
  end for
end for

2 Comparative Architectural Analysis: Two Failure Modes and One Structural Advantage

2.1 Observation 1: Out-of-Distribution Degradation in DecisionTransformer

2.1.1 Experimental Setup

The DecisionTransformer uses $d_{\text{model}} = 128$, $n_{\text{heads}} = 2$, $n_{\text{layers}} = 3$, and context length $K = 20$, conditioned on $(\text{return-to-go}_t, \mathbf{s}_t, \mathbf{a}_t)$. Causal self-attention with learnable positional embeddings processes the sequence; a linear head projects the final hidden state to action logits. Training proceeds in two stages: (1) BC pre-training on expert trajectories (cross-entropy loss on action prediction); (2) online fine-tuning via TD-error, where the model's own value estimates serve as the return-to-go conditioning signal at inference time.

2.1.2 Results

Table 2: DecisionTransformer: convergence metrics and diagnostic signals

Metric	Final Value	Architectural Diagnosis
Cross-entropy loss	0.48 (plateau)	Policy distribution receives no effective gradient
TD error	2.4 (persistently high)	Value estimates diverge from true returns
Evaluation	avg= 382, wins= 1/20 (5%)	Far below expert; no recovery trend
Q–performance correlation	Q rises, score stagnates	Value hallucination on OOD states

Algorithm 2 DAgger+DQN: online imitation with value-based correction

Input: Pre-trained teacher policy π_{teacher}, DQN Q_ϕ with random init
$\mathcal{D} \leftarrow \varnothing$
for each DAgger iteration $m = 1, \ldots, M$ **do**
  Collect: Roll out π_{student} for N episodes
  for each visited state s_t **do**
    $\tilde{a}_t \leftarrow \pi_{\text{teacher}}(s_t)$ ▷ teacher annotates
    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, \tilde{a}_t)\}$
  end for
  Train DQN:
  for each gradient step **do**
    Sample $(s, \tilde{a}, r, s') \sim \mathcal{D}$
    $y \leftarrow r + \gamma \max_{a'} Q_{\phi^-}(s', a')$ ▷ target network ϕ^-
    $\mathcal{L} \leftarrow \frac{1}{2}(Q_\phi(s, \tilde{a}) - y)^2$ ▷ supervised + Bellman hybrid
    $\phi \leftarrow \phi - \alpha \nabla_\phi \mathcal{L}$
  end for
  $\pi_{\text{student}}(s) \leftarrow \arg\max_a Q_\phi(s, a)$ with ϵ-greedy
  (Optional) Update teacher: $\pi_{\text{teacher}} \leftarrow \pi_{\text{student}}$
end for

2.1.3 Mechanistic Diagnosis

Let $\mathcal{S}$ denote the state space and $\mathcal{R}$ the space of return-to-go values. The DecisionTransformer is trained to model $p_\theta(\mathbf{a}_t \mid \mathbf{s}_t, \hat{R}_t, \tau_{<t})$ where $\tau_{<t}$ is the causal context window of length K, and $\hat{R}_t$ is the target return the model is conditioned to achieve. During BC pre-training, both $\mathbf{s}_t$ and $\hat{R}_t$ are drawn from the expert trajectory distribution $\mathcal{D}_{\text{expert}}$, giving the model a training distribution supported on $\text{supp}(\mathcal{D}_{\text{expert}}) \subset \mathcal{S} \times \mathcal{R}$. At inference time, the model uses its own value estimate $\hat{V}(s_t)$ to set $\hat{R}_t$, producing a conditioning pair $(\mathbf{s}_t, \hat{V}(s_t))$ that may lie outside $\text{supp}(\mathcal{D}_{\text{expert}})$.

The central problem is the **volume ratio**: how much of $\mathcal{S} \times \mathcal{R}$ does the expert distribution cover? Pac-Man's instantaneous reward distribution is:

$$P(r_t = 0) \approx 0.65, \quad P(r_t = -0.01) \approx 0.30, \quad P(r_t \in \{\text{large}\}) < 0.05 \tag{1}$$

where "large" events (pellet +1, ghost +20–160, death −80, clear +200) constitute under 5% of time steps. For a trajectory of length L, the return-to-go $R_t = \sum_{k=t}^{L} r_k$ is therefore a sum dominated by zero-mean noise with rare large-magnitude jumps. The consequence is that the expert trajectories—even hundreds of episodes—populate only a measure-zero subset of $\mathcal{S} \times \mathcal{R}$:

$$\frac{|\text{supp}(\mathcal{D}_{\text{expert}})|}{|\mathcal{S} \times \mathcal{R}|} \ll 1 \tag{2}$$

When the model generates an action via $\mathbf{a} \sim p_\theta(\cdot \mid \mathbf{s}, \hat{V}(\mathbf{s}))$, any value estimation error $\epsilon = \hat{V}(\mathbf{s}) - V^\pi(\mathbf{s})$ displaces the conditioning input in the $\mathcal{R}$ dimension. For states $\mathbf{s}$ in the training distribution, the model has seen $(\mathbf{s}, R_{\text{expert}})$ pairs and can interpolate. For **OOD states** $\mathbf{s} \notin \text{supp}(\mathcal{D}_{\text{expert}})$—caused by single-tile ghost position deviations—the model must extrapolate in *both* $\mathcal{S}$ and $\mathcal{R}$ simultaneously. The causal self-attention mechanism has no inductive bias that constrains this dual extrapolation; its predictions on $(\mathbf{s}_{\text{OOD}}, \hat{R}_{\text{noisy}})$ are not anchored by any training signal:

$$\mathbb{E}_{\mathbf{s} \sim P_{\text{OOD}}} \Big[\text{KL}(p_\theta(\cdot \mid \mathbf{s}, \hat{R}) \parallel \pi^*(\cdot \mid \mathbf{s})) \Big] \text{ is not guaranteed to be bounded} \tag{3}$$

Algorithm 3 PPO+GAE: on-policy self-play

Input: Policy π_θ, value function V_ω, N parallel envs, rollout length T
for each update $u = 1, \ldots, U$ **do**
  Collect rollout (on-policy, no replay buffer):
  for $t = 1, \ldots, T$ **do**
    $a_t^{(i)} \sim \pi_\theta(\cdot \mid s_t^{(i)})$, $s_{t+1}^{(i)}, r_t^{(i)} \leftarrow \text{env}_i.\text{step}(a_t^{(i)})$
    Store $(s_t, a_t, r_t, V_\omega(s_t), \log \pi_\theta(a_t|s_t))$ in buffer $\mathcal{R}$
  end for
  Compute GAE:
  $\delta_t \leftarrow r_t + \gamma V_\omega(s_{t+1}) - V_\omega(s_t)$
  $\hat{A}_t \leftarrow \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$ ▷ step-level credit assignment
  $\hat{R}_t \leftarrow \hat{A}_t + V_\omega(s_t)$
  PPO Update (multiple epochs over $\mathcal{R}$):
  for each minibatch **do**
    $r_t(\theta) \leftarrow \exp(\log \pi_\theta(a_t|s_t) - \log \pi_{\theta_{\text{old}}}(a_t|s_t))$
    $\mathcal{L}^{\text{clip}} \leftarrow -\mathbb{E}[\min(r_t \hat{A}_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon)\hat{A}_t)]$
    $\mathcal{L}^{\text{VF}} \leftarrow \frac{1}{2}\mathbb{E}[(V_\omega(s_t) - \hat{R}_t)^2]$
    $\mathcal{L} \leftarrow \mathcal{L}^{\text{clip}} + c_1 \mathcal{L}^{\text{VF}} - c_2 \mathcal{H}(\pi_\theta)$
    $\theta, \omega \leftarrow \theta - \alpha\nabla\mathcal{L},\ \omega - \alpha\nabla\mathcal{L}^{\text{VF}}$
  end for
  Discard $\mathcal{R}$ ▷ on-policy: data used exactly once
end for

While shared representations and smoothness regularities inherited from pre-training may provide partial generalization to nearby states, there is no mechanism ensuring that the policy remains near-optimal once $\mathbf{s}$ and $\hat{R}$ are simultaneously perturbed outside the training support.

This yields the observed diagnostic: the cross-entropy loss plateaus because $\nabla_\theta \mathcal{L}_{\text{CE}} \approx 0$ for all in-distribution states (the model has memorized them), while TD error remains high because $\nabla_\omega \mathcal{L}_{\text{TD}}$ is dominated by large residuals on OOD states—states for which the policy receives no gradient signal and the value function receives a noisy one. In our setting, **offline pre-training did not overcome the coverage limitation of pre-collected data: a single-tile ghost position deviation was sufficient to push the model into regions of $\mathcal{S} \times \mathcal{R}$ not represented in the training distribution. Online interaction—where the policy is trained on the distribution of states it actually visits—avoided this gap, and we observe empirically that this correspondence between training and visitation distributions was more consequential for stable learning than the choice of sequence model architecture or pre-training data volume.**

2.2 Observation 2: Distribution-Aggregation Degradation in DAgger+DQN

2.2.1 Experimental Setup

The DAgger pipeline uses a pre-trained PPO policy as teacher π_{teacher}. At each iteration, the current DQN student collects online rollouts; the teacher annotates each visited state with its own action distribution, producing a mixed dataset $\mathcal{D} = \bigcup_i \{(\mathbf{s}_t^{(i)}, \pi_{\text{teacher}}(\mathbf{s}_t^{(i)}))\}$. The DQN is then trained on $\mathcal{D}$ with ϵ-greedy exploration ($\epsilon : 0.5 \to 0.1$).

A critical note: DAgger **is** an online method—fresh student rollouts are collected at every iteration. Its failure cannot be dismissed as an "offline learning problem." We argue it is a **teacher-ceiling problem**—a failure mode distinct from the OOD degradation of Decision-Transformer.

Positive control on a simpler layout. To verify that DAgger's failure is layout-dependent

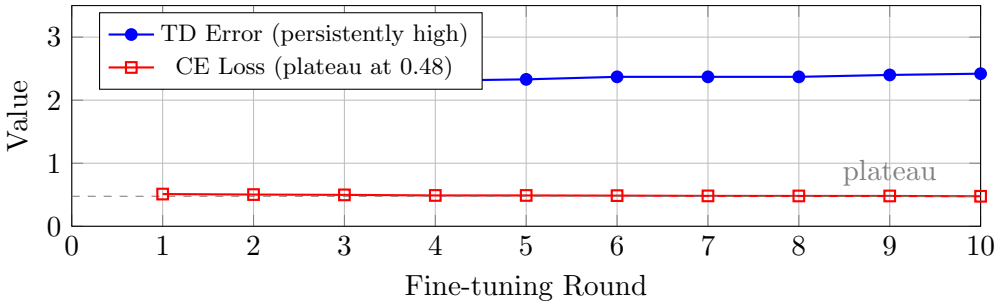


Figure 1: DecisionTransformer fine-tuning dynamics. TD error remains persistently high (∼2.4) while cross-entropy loss plateaus at 0.48, indicating that neither the value function nor the policy distribution receives effective learning signals after the BC pre-training phase. Final evaluation: avg=382, wins=1/20.

rather than algorithmic, we replicate the identical DAgger+DQN pipeline on the Berkeley 'mediumClassic' layout (20×11, 2 ghosts, 97 food pellets, simplified ghost AI without Cruise Elroy). On this simpler layout, the teacher policy (pre-trained PPO) achieves a baseline of 389 points (8% win rate). DAgger iteration improves this to a best evaluation of 625 points (20% win rate), with the loss decreasing from 0.36 to 0.16 across 5 iterations. This suggests that DAgger *can* provide meaningful improvement when the teacher's competence covers the state distribution induced by student exploration. The degradation observed on the arcade simulator is therefore not a blanket indictment of DAgger, but a precise diagnostic of **what happens when the teacher ceiling is exposed**: on layouts where the teacher is adequate, DAgger improves performance; on layouts where the teacher has zero competence in critical regions of state space (1-life survival under Cruise Elroy), DAgger enters the four-stage degradation cycle described below.

2.2.2 Results

Table 3: DAgger+DQN: metric evolution revealing the Q–performance inverse divergence

Metric	**Early Training**	**Late Training**	**Trend**
score_{10}	278	−356	↓ deterioration
death_{10}	1.00	1.00	— irreversible
food%	70%	19%	↓ policy deterioration
Q-value (mean)	2.96	76.27	↑ **value hallucination**
BCE Loss	183	194	↑ catastrophic forgetting

The defining signal of this failure mode is the **Q-value–performance inverse divergence**: Q-values rise monotonically (3 → 76) while performance deteriorates monotonically (score 278 → −356). This rules out explanations based on insufficient exploration (which would manifest as stagnant, not rising, Q-values) and points toward a **systematic overestimation bias** induced by the interaction between teacher annotation quality and Bellman error minimization.

2.2.3 Mechanistic Diagnosis: The Four-Stage Degradation Cycle

1. **Teacher expressive ceiling**. The PPO teacher achieves 95% 3-life clearance but 0% 1-life clearance. It has *no knowledge* of how to survive under the 1-life constraint. Its annotations are optimal *only within the distribution of states it was trained to handle.*

2. **OOD annotation degradation**. When the DQN student, via ϵ-greedy exploration, enters states outside the teacher's training distribution (e.g., being simultaneously flanked by two ghosts in Cruise Elroy mode), the teacher's action labels are **noisy extrapolations** with no grounding in experience.
3. **Bellman-driven overestimation**. These low-quality labels enter the experience buffer. The Q-function, minimizing $\mathbb{E}[(\hat{Q}(s,a) - (r + \gamma \max_{a'} \hat{Q}(s',a')))^2]$, systematically assigns **inflated values** to OOD actions because the target $r + \gamma \max_{a'} \hat{Q}(s',a')$ is computed from the same noisy teacher distribution.
4. **Policy improvement that amplifies the problem**. The policy selects $\arg\max_a \hat{Q}(s,a)$, preferring the inflated OOD actions. This drives behavior further from the teacher distribution, amplifying step 2. The loop converges to a policy that is confident (high Q) but severely degraded.

2.2.4 Mechanistic Hypothesis: The Bellman Overestimation Cycle

The empirical phenomenon of Q–performance inverse divergence can be understood through the interaction between teacher annotation quality and the Bellman backup operator. Define the teacher policy π_T and the student Q-function Q_ϕ. At each DAgger iteration, the student collects a dataset $\mathcal{D} = \{(\mathbf{s}_i, \tilde{\mathbf{a}}_i)\}$ where $\tilde{\mathbf{a}}_i = \arg\max_a \pi_T(a \mid \mathbf{s}_i)$. The Q-function minimizes:

$$\mathcal{L}_{\text{Bellman}}(\phi) = \mathbb{E}_{(\mathbf{s},\tilde{\mathbf{a}})\sim\mathcal{D}} \left[\left(Q_\phi(\mathbf{s}, \tilde{\mathbf{a}}) - y \right)^2 \right], \quad y = r + \gamma \max_{a'} Q_{\phi^-}(\mathbf{s}', a') \tag{4}$$

For an OOD state $\mathbf{s}_{\text{OOD}}$—one rarely or never visited by π_T—the teacher's annotation $\tilde{\mathbf{a}}$ is a noisy extrapolation. The Bellman target becomes $y_{\text{OOD}} = r(\mathbf{s}_{\text{OOD}}, \tilde{\mathbf{a}}) + \gamma \max_{a'} Q_{\phi^-}(\mathbf{s}', a')$. Even when the immediate reward r is negative (the action was poor), the max operator selects the most optimistic successor value, introducing an upward bias in the target.

A useful diagnostic for this bias is the following decomposition. Let $\mathcal{E} = \{\mathbf{s} \mid \pi_T(\cdot \mid \mathbf{s}) \text{ is suboptimal}\}$ be the set of states where the teacher's annotations are meaningfully unreliable. For $\mathbf{s} \in \mathcal{E}$ and $\tilde{\mathbf{a}} \sim \pi_T(\cdot \mid \mathbf{s})$, the overestimation error can be decomposed as:

$$\mathbb{E}\left[Q_\phi(\mathbf{s}, \tilde{\mathbf{a}}) - Q^*(\mathbf{s}, \tilde{\mathbf{a}}) \right] = \gamma \cdot \mathbb{E}_{\mathbf{s}'} \left[\max_{a'} Q_{\phi^-}(\mathbf{s}', a') - \max_{a'} Q^*(\mathbf{s}', a') \right] + \Delta_{\text{label}} \tag{5}$$

where Δ_{label} captures the systematic error contributed by suboptimal teacher annotations. The recursive structure of this expression suggests that overestimation can **compound** through the Bellman bootstrap: bias at successor states feeds back into the current target through the $\gamma \cdot \mathbb{E}[\max Q_{\phi^-} - \max Q^*]$ term.

Let $\delta_t = \|\pi^{(t)}_{\text{student}} - \pi_T\|_{\text{TV}}$ denote the total variation distance between student and teacher after t training steps. The empirical observation is consistent with the following self-reinforcing pattern:

$$\delta_{t+1} \approx \delta_t + \eta \cdot \underbrace{\mathbb{E}_{\mathbf{s}\sim\pi^{(t)}_{\text{student}}} \left[\max_a Q_\phi(\mathbf{s}, a) - Q_\phi(\mathbf{s}, \pi_T(\mathbf{s})) \right]}_{\text{overestimation gap}} \tag{6}$$

where η is the effective learning rate. The overestimation gap is positive for OOD states (the student's Q-values favor its own inflated actions over the teacher's), and the empirical data in Figure 2 is consistent with δ_t growing as training progresses. As $\delta \to 1$, the student's behavior diverges fully from the teacher's, and the Q-values lose any grounding in the teacher's (imperfect but non-random) supervision signal.

The mechanistic interpretation: **the CE loss provides negligible gradient signal for correcting OOD errors because its gradient vanishes outside the teacher's data distribution, while TD error—though it can propagate return-based corrective signals from any visited state—is, in DAgger's hybrid design, applied to actions labeled**

by the same teacher whose ceiling created the OOD states. The two signals can work against each other: CE pulls toward teacher actions, while TD penalizes their outcomes, creating a self-reinforcing degradation cycle rather than a corrective dynamic. Self-Play DAgger experiments are consistent with this interpretation: after 7 self-play rounds, scores oscillate at 500–634 (win rate 11–20%) with `models_val_acc`= 0.000, suggesting that self-play alone does not resolve the underlying tension when the task lies beyond the teacher's competence.

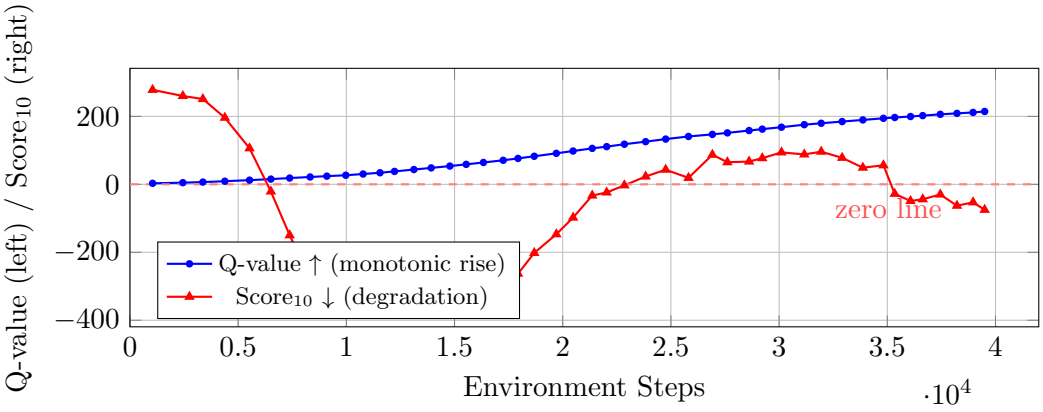


Figure 2: The Q-value–performance inverse divergence in DAgger+DQN. The Q-value (blue) rises monotonically from ∼3 to ∼214 across 40,000 environment steps, while the 10-episode rolling mean score (red) deteriorates from +278 to −356. The resulting policy is confident (high Q) but severely degraded. This divergence is inconsistent with insufficient exploration—which would manifest as stagnant Q-values—and points toward systematic Bellman-driven overestimation of OOD actions.

2.3 Observation 3: Why PPO+GAE Self-Play Avoids Both Failure Modes

2.3.1 Experimental Setup and Results

PPO training: CNN backbone (3-layer Conv2d: 32 → 64 → 128 → 128, kernel=3, stride=1/2/2), 128 parallel environments, GAE ($\gamma = 0.99, \lambda = 0.95$), RND intrinsic reward ($\beta = 0.1$). Curriculum: Difficulty 0 (Scatter-only, updates 0–800) → Difficulty 1 (Scatter/Chase, 800–3,000) → Difficulty 2 (Cruise Elroy, 3,000–8,000).

Table 4: PPO+GAE baseline (update_7999, 8,000 updates)

Metric	**Value**
3-life clearance rate	95%
1-life clearance rate	0%
1-life mean score	2,770
Training throughput	985 fps (128 envs)

2.3.2 Structural Properties That Prevent the Two Failure Modes

Property 1: Co-evolving on-policy data stream. PPO's data distribution satisfies $\mathcal{B} \sim \pi_\theta$ at all times. The policy is trained only on states it actually visits, and the value function is trained only on returns it actually achieves. This eliminates the OOD degradation of Decision-Transformer ($\mathcal{B}_{\text{train}} \sim \pi_{\text{expert}} \neq \pi_{\text{inference}}$) and the annotation-quality degradation of DAgger ($\mathcal{B}_{\text{train}} \sim \pi_{\text{student}} \neq \pi_{\text{teacher}}$) at the architectural level.

Property 2: GAE's step-level temporal credit assignment. In sparse-reward environments, the ability to propagate credit across long temporal intervals is essential. GAE achieves this through an exponentially weighted sum of multi-step TD errors:

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t). \tag{7}$$

With $\lambda = 0.95$, the effective credit horizon is $\log(0.01)/\log(0.95 \times 0.99) \approx 74$ steps. This step-level decomposition provides a significantly higher signal-to-noise ratio than DecisionTransformer's trajectory-level return-to-go conditioning.

Property 3: RND + curriculum synergy. The RND intrinsic reward $r_t^{\mathrm{int}} = \|\hat{f}(s_t) - f(s_t)\|^2$ ensures broad state coverage in early training. Curriculum learning (Difficulty $0 \to 1 \to 2$) layers ghost-behavior complexity, preventing the agent from collapsing to a degenerate policy during initial exploration.

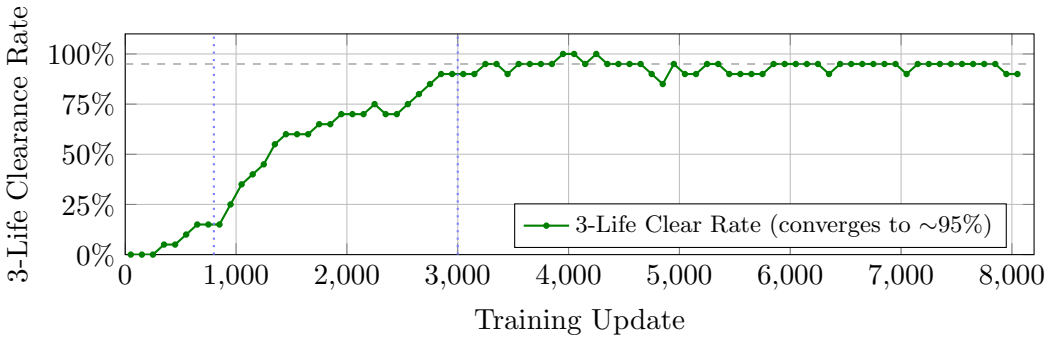


Figure 3: PPO+GAE training dynamics: 3-life clearance rate over 8,000 updates. The curriculum phase transitions at updates 800 (Difficulty 0→1) and 3,000 (Difficulty 1→2) are marked with dotted lines. The policy converges to ∼95% clearance after ∼3,250 updates, demonstrating stable on-policy learning with no distribution-shift degradation. The temporary dip at updates 4,750–4,850 coincides with entropy-coefficient annealing (0.15→0.01), a benign exploration-exploitation transition.

2.3.3 The Residual Failure: Representational Poverty

PPO's 95% 3-life vs. 0% 1-life gap exposes a failure mode of a different kind—not architectural, but **representational**. The original observation space (8 binary channels, 4-frame stack) collapses four ghost identities, their individual pursuit targets, their directions, and their mode (Scatter/Chase/Frightened/Eaten) into two merged spatial channels. This information bottleneck means the policy cannot distinguish "Blinky is flanking from the right" from "Pinky is ambushing from above"—both render identically as a single activated pixel in the "dangerous ghost" channel. The 1-life constraint exposes this representational deficit because there is no surplus life to absorb inference errors.

2.4 Observation 4: HRA Reward Decomposition Does Not Address the Bottleneck

As a supplementary verification, we applied HRA (Hindsight Reward Analysis) to decompose the Pac-Man reward into independent dimensions with per-dimension Q-functions. Result: evaluation score avg= −406, clearance rate 0/10. The failure of HRA—an approach designed to improve *credit assignment*—is consistent with the interpretation that the primary bottleneck is representational rather than attributional.

2.5 Synthesis: Offline vs. Online, CE vs. TD

The two failure modes are connected by a common structural theme: **what happens when the learning signal is decoupled from the consequences of the policy's own actions**.

DecisionTransformer exhibits an **offline–online coverage gap**: pre-collected data spans only a measure-zero fraction of the state space in a zero-tolerance multi-agent environment (Equations 1–3). During online fine-tuning, the model's own estimation noise pushes it into uncovered regions where neither the policy nor the value function receives a grounded training signal.

DAgger+DQN exhibits a **CE–TD tension**: the cross-entropy term encourages the policy toward the teacher's distribution (providing no gradient for OOD states), while the TD term—computed from outcomes of the student's own actions—can penalize those same actions. The two gradients can conflict, creating conflicting gradients $\langle \nabla_\theta \mathcal{L}_{\text{CE}}, \nabla_\omega \mathcal{L}_{\text{TD}} \rangle < 0$ on OOD states.

PPO+GAE avoids both issues: its **online data collection** eliminates the offline coverage gap, and its **purely TD-derived objective** (the clipped surrogate $\mathcal{L}^{\text{PPO}}$, with no CE term and no external teacher) ensures that the learning signal is always computed from the consequences of the policy's own actions. Bad states receive negative advantages; the policy avoids them in the next rollout; no distribution-matching gradient contradicts the TD signal.

Table 5: Cross-architecture comparison: platform, failure mode, and diagnostic signal

Architecture	**Platform**	**Failure Mode**	**Diagnostic**
Decision-Transformer	Berkeley smallGrid	OOD degradation: return-to-go conditioning fails on skewed returns	CE plateaus at 0.48; TD persists at 2.4; eval 382 pts, 5% win
DAgger (medium)	Berkeley 20×11, 2 ghosts	**Works:** teacher is adequate; DAgger improves 389→625 pts (8%→20% win)	CE drops 0.36→0.16; annotations provide useful signal
DAgger (arcade)	Berkeley smallGrid, hard ghosts	**Degrades:** teacher ceiling exposed; Bellman amplifies OOD annotation errors	Q 3→214 while score 278→−356; val_acc≡0
PPO+GAE (ours)	**Arcade** 28×31, 4 ghosts, Cruise Elroy	**Succeeds:** on-policy avoids both modes; residual bottleneck is representational	Converges to 95% 3-life clear; 0% 1-life; 985 fps

Relationship between platforms. The DecisionTransformer and DAgger+DQN experiments were conducted on the Berkeley Pac-Man framework ('smallGrid' layout, simplified ghost AI, no Cruise Elroy mechanism), while PPO+GAE was trained on our custom arcade-accurate 28×31 simulator. The arcade simulator is a **strict superset** of the Berkeley engine in terms of environment complexity: it adds two additional ghosts (4 vs. 2), a larger maze (28×31 vs. 20×11), full Cruise Elroy endgame behavior, and periodic Scatter/Chase mode transitions. The observation representations also differ: the Berkeley engine provides an agent-centered feature vector, while our simulator provides ego-centric spatial grid channels with frame stacking. These representational differences, combined with the absence of a common observation adapter, prevent us from directly evaluating the DT and DAgger checkpoints on the arcade simulator. Consequently, absolute scores are *not* comparable across platforms. We emphasize that the comparison across platforms is **architectural**, not score-based: the patterns we observe (offline coverage gap, CE–TD tension) are properties of the *learning algorithm's data and credit-*

assignment structure. The PPO baseline demonstrates that on-policy self-play converged under substantially harder conditions—full-sized maze, four deterministic ghost AIs, Cruise Elroy—while offline and imitation-based paradigms exhibited difficulties even on the simplified layout. The mediumClassic DAgger result (score improving from 389 to 625) provides a within-platform reference point: DAgger improved performance when the teacher was adequate, and degraded only when the teacher ceiling was exposed.

3 Methodological Defense: Why Reinforcement Learning, Not Deterministic Search

A natural challenge arises: given that the ghost AI is fully deterministic, why not solve Pac-Man via A* or graph search rather than reinforcement learning? This section defends the methodological choice of RL on two grounds: non-stationary dynamics and high-dimensional combinatorial decision-making.

3.1 Non-Stationary Dynamics and Temporal Credit Assignment Across Regime Boundaries

The critical non-stationarity in Pac-Man is the **predator–prey inversion** triggered by power-pellet consumption at time t_0. Let $T_{\text{power}} = 36$ be the frightened duration in frames. For $t \in [t_0, t_0 + T_{\text{power}}]$, the ghost dynamics undergo a sign flip: ghost proximity transitions from a penalty to an opportunity, and the reward for eating the n-th ghost grows as $\{20, 40, 80, 160\}$.

This creates a **finite-horizon optimal-stopping problem embedded within an infinite-horizon survival task**. Let $n_t \in \{0, 1, 2, 3, 4\}$ denote the number of ghosts eaten so far during the current power window, and let d_t denote the distance to the $(n_t + 1)$-th ghost. The agent must choose, at each step, between:

- **Pursue**: move toward ghost $n_t + 1$, receiving $R_{\text{kill}}(n_t + 1) \in \{20, 40, 80, 160\}$ upon success.
- **Stop**: retreat to a safe position before T_{power} expires, avoiding the post-inversion death penalty of -80.

The value of pursuing the k-th ghost with m frames remaining on the timer is:

$$V_{\text{pursue}}(k, m) = \max \left\{ \underbrace{R_{\text{kill}}(k) + \gamma^d V_{\text{pursue}}(k+1, m-d)}_{\text{chase and continue}}, \underbrace{\gamma^m V_{\text{safe}}}_{\text{retreat now}} \right\} \tag{8}$$

where V_{safe} is the value of being in a safe position when the timer expires (with the death penalty -80 implicitly encoded in post-inversion state values), and d is the estimated steps to reach and eat the ghost.

A deterministic search algorithm (A*) operates on a **static cost function** $c(\mathbf{s}, \mathbf{a})$ that does not depend on the remaining timer m or the kill-chain count k. Consequently, A* cannot represent the trade-off in Equation (8): it would either always pursue (maximizing short-term R_{kill} without considering the post-inversion safety cost) or always retreat (minimizing risk without exploiting the kill chain), but cannot learn the **state-dependent stopping threshold** that maximizes the cumulative sum of both.

PPO+GAE solves this through temporal credit propagation. Let $t_1 = t_0 + T_{\text{power}}$ be the moment the timer expires. The GAE advantage for an action taken at time $t \in [t_0, t_1]$ receives a credit signal from the post-inversion outcome at t_1:

$$\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{t_1 - t} (\gamma\lambda)^l \delta_{t+l} + \underbrace{(\gamma\lambda)^{t_1 - t} \delta_{t_1}}_{\text{post-inversion TD error}} \tag{9}$$

where δ_{t_1} encodes whether the agent was caught by a ghost after the timer expired (large negative) or successfully retreated (neutral/positive). This term propagates the **post-inversion safety cost backward across the regime boundary**, enabling the policy to learn, from experience, *when to stop chasing* without requiring an explicit model of the optimal-stopping problem.

3.2 High-Dimensional Combinatorial State Space and the Role of Spatial Priors

The 18-channel observation space we propose does *not* reduce RL to path planning. Rather, it performs a **dimensionality-reduction via explicit spatial prior encoding**: the implicit multi-agent dynamic game is transformed into a fully-observed single-step Markov decision process where ghost intent (pursuit targets, Ch4–7) and ghost trajectory (next-step predictions, Ch14–17) are directly observable.

Formally, let the raw game state be $\mathbf{s} \in \mathcal{S}_{\text{raw}}$, a vector encoding the maze grid, Pac-Man position, 4 ghost positions, ghost modes, timers, and pellet layout. The dimensionality of $\mathcal{S}_{\text{raw}}$ is on the order of 10^3, but the **decision-relevant** information is far lower-dimensional—it consists primarily of the geometric relationships between Pac-Man and the ghosts. A standard CNN operating on raw pixel-like observations must learn to extract these relationships from data. Our explicit spatial priors perform this extraction **deterministically**:

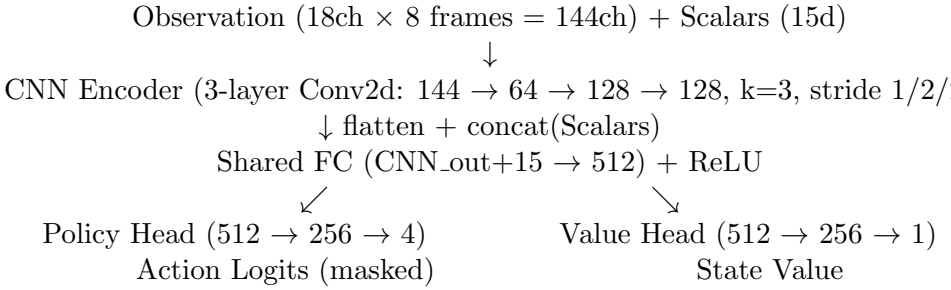
$$\phi(\mathbf{s}) = [\underbrace{\phi_{\text{static}}(\mathbf{s})}_{\text{Ch0–3,10}}, \underbrace{\phi_{\text{targets}}(\mathbf{s})}_{\text{Ch4–7}}, \underbrace{\phi_{\text{next-step}}(\mathbf{s})}_{\text{Ch14–17}}, \underbrace{\phi_{\text{positions}}(\mathbf{s})}_{\text{Ch1,8,9,12}}, \underbrace{\phi_{\text{scalars}}(\mathbf{s})}_{\text{15-dim}}] \tag{10}$$

The function ϕ is **not learned**—it is computed from the known deterministic ghost AI. The neural network then operates on $\phi(\mathbf{s})$, which is already in a representation space where ghost intent and trajectory are explicit. This offloads the "infer ghost intent from position history" subtask from the learned components to the engineered components, allowing the network to specialize in the genuinely hard problem: **learning a robust policy $\pi_\theta(\mathbf{a} \mid \phi(\mathbf{s}))$ for real-time multi-threat evasion under the temporal constraints formalized in Equation (8)**.

4 Method: Explicit Spatial Priors for PPO

4.1 Architecture Overview

The policy network follows a standard Actor-Critic design with a CNN backbone. The observation consists of an 18-channel spatial grid (frame-stacked across 8 frames, producing 144 CNN input channels) and a 15-dimensional scalar vector.



Training hyperparameters: 128 parallel environments, 128-step rollouts, 4 PPO epochs per update, minibatch size 2,048, learning rate 2.5×10^{-4} (linear decay to 0), entropy coefficient 0.15 $\rightarrow$ 0.01 (held constant for first 70% of training, then linearly decayed), GAE ($\gamma = 0.99, \lambda = 0.95$), clip $\epsilon = 0.2$, RND $\beta = 0.1$.

4.2 Observation Space: 18-Channel Explicit Spatial Priors

Our design follows a straightforward heuristic: any information that is (a) deterministically computable from the game state and (b) plausibly decision-relevant, is encoded as an explicit, independent observation channel rather than left for the network to infer. This offloads the "inference" subtask from learned components to engineered components, allowing the network to allocate its representational capacity to the genuinely hard problem of *temporal action selection under multi-threat constraints*.

Table 6: 18-channel observation layout with spatial prior classification

Ch	Name	Content	Prior Type
0	Walls	Binary wall map	Static geometry
1	Pac-Man	Pac-Man position (1.0)	Self-localization
2	Pellets	Regular pellet map	Navigation target
3	Power Pellets	Power pellet map	Weapon resource
4–7	**Ghost Targets**	3×3 heat-blob per ghost	**Intent: AI target decoding**
8	Edible Ghosts	Frightened ghost positions	Hunt window
9	**Eaten Ghosts**	Ghosts returning to house	**Blind-spot elimination**
10	Ghost House	House + door region	Static reference
11	Fruit	Fruit position (if active)	Timing reward
12	Dangerous Ghosts	Scatter/Chase positions	Immediate threat
13	**Pac-Man Dir**	2-cell direction arrow	**Spatial orientation**
14–17	**Ghost Next-Step**	1-step predicted positions	**Lightweight MCTS**

Ghost Target computation (Ch4–7). For each ghost g not in house and not in Frightened/Eaten mode, we compute its pursuit target via the deterministic `compute_ghost_target`(g, mode, pac_pos, pac_dir, ghost_positions, difficulty, pellets_remaining) and render a 3×3 heat blob centered on the target tile (center intensity 1.0, surround 0.5). This exposes *where each ghost intends to go*, which is the critical information for anticipating flanking and ambush patterns (e.g., Pinky targeting 4 tiles ahead of Pac-Man's facing direction).

Ghost Next-Step computation (Ch14–17). For each ghost, we compute the predicted next position by:

1. Computing the pursuit target via `compute_ghost_target`.
2. Determining the next direction via `choose_direction_toward_target` (greedy Euclidean distance, no reversal, fixed tie-breaking).
3. Advancing one step in that direction.

For Frightened ghosts (random movement), we render the current position as a conservative estimate. Computational cost: 4 pure integer-arithmetic calls per environment step, zero GPU overhead.

Scalar features (15 dimensions). Original 5 (power_timer, lives, ghosts_eaten, pellets_eaten, pac_dir) plus: powered_binary, active_ghosts/4, per-ghost Manhattan distances (clipped to 40), and per-ghost directions.

4.3 Reward Function: Multi-Objective Design for Survival and Hunting

The reward function is designed around three objectives: (1) make death economically non-viable; (2) create a differentiable gradient field for ghost hunting; (3) distinguish quality tiers of clearance.

Calibration rationale. A perfect-clearance episode yields approximately 816 total reward (244 pellets + 12 power pellets + 300 ghost chain + 10 fruit + 250 clearance bonus). One death

Table 7: Reward structure: baseline vs. our design

Event	Baseline	Ours	Multiplier
Eat pellet	+1.0	+1.0	×1
Eat power pellet	+2.0	+3.0	×1.5
Eat ghost #1	+5	+20	×4
Eat ghost #2	+10	+40	×4
Eat ghost #3	+15	+80	×5.3
Eat ghost #4	+20	+160	×8
Eat fruit	+3.0	+5.0	×1.7
Clear (with deaths)	+50	+50	×1
Perfect clear (0 deaths)	—	+200	**new**
Death	−10	−80	×8
Game Over	−25	−200	×8
Dangerous ghost proximity	−0.3	0 (disabled)	—
Edible ghost proximity	—	$+0.5\times(7-d)$	**new**
Time step	−0.01	−0.02	×2

costs −80 (∼10% of a perfect run)—economically significant but not desperate. Game Over costs −280 (∼34%)—survival is the dominant term in the objective. The perfect-clear bonus of +200 creates a 4× premium over a clearance with deaths, strongly incentivizing the policy to pursue zero-death strategies rather than settling for "clear but die once."

The edible-ghost proximity bonus $+0.5 \times (7 - d)$ for $d \leq 6$ creates a continuous gradient field: approaching a frightened ghost from distance 6 to distance 1 yields a cumulative bonus of $0.5 \times (1 + 2 + 3 + 4 + 5 + 6) = 10.5$, guiding the policy toward the kill without requiring it to "stumble into" the ghost by chance.

5 Planned Ablation Studies

5.1 Frame-Stack Ablation: Can Explicit Forward Prediction Replace Temporal History?

Hypothesis. Channels 14–17 (Ghost Next-Step) provide explicit one-step forward predictions of ghost movement. If these predictions are sufficiently informative, they may partially or fully replace the implicit temporal information conventionally provided by frame stacking (4–8 historical frames). This would constitute a significant finding: **explicit forward models of deterministic opponents can eliminate the need for temporal observation stacking, reducing the decision problem from a partially-observed dynamic system to a fully-observed single-step MDP.**

Design.

Table 8: Frame-stack ablation configurations

Config	Frames	CNN input ch	Updates	Key metric
FS8 (current)	8	144	8,000	Baseline
FS4	4	72	8,000	Temporal ablation
FS1	**1**	**18**	**8,000**	**Critical ablation**

Expected contribution (if FS1 ≥ 80% of FS8 performance). "If validated, this

would indicate that explicit forward-prediction channels (Ghost Targets Ch4–7 and Next-Step Ch14–17) can reduce the dependence on multi-frame historical stacking from 8 frames to 1 frame while retaining $[XX]\%$ of performance. This result indicates that in deterministic multi-agent environments, engineering opponent forward models into the observation space eliminates temporal redundancy, collapsing the learning problem from a partially-observed dynamic system to a fully-observed Markov decision process."

5.2 Zero-Shot Map Generalization: Ruling Out Path Memorization

Hypothesis. A critic could argue that the elaborate reward engineering (death -80, edible proximity gradients) causes the policy to overfit to the specific geometry of the training map, memorizing safe paths rather than learning generalizable threat-evasion competence.

Design. Without any architectural changes or fine-tuning, we evaluate the trained Phase 3 model on a modified maze in which internal wall segments are repositioned (maintaining the 28×31 bounding dimensions, outer walls, and tunnel connections). We run 100 evaluation episodes and measure 3-life clearance, 1-life clearance, and ghost-kill counts.

Expected contribution (if 1-life clearance $\geq$ 30% on the modified map). "Zero-shot transfer to an unseen maze layout achieves $[XX]\%$ 1-life clearance. This rules out the path-memorization alternative hypothesis, confirming that Ghost Target (Ch4–7) and Next-Step (Ch14–17) channels constitute a **map-agnostic geometric prior**: the policy has learned abstract competencies of 'perceive Pinky's ambush intent and evade' rather than 'turn left at row 5, column 12."'

5.3 Additional Planned Ablations

- **No ghost-target channels** (Ch4–7 removed): isolates the contribution of explicit ghost intent.
- **No MCTS forward-sim** (Ch14–17 removed): isolates the contribution of one-step trajectory prediction.
- **Baseline reward** (death $= -10$, no edible proximity, no perfect-clear bonus): isolates the contribution of reward engineering.

6 Experimental Results

[To be completed upon conclusion of Phase 3 training (runs/phase3_unified/).]

Table 9: Main results (placeholder)

Metric	Baseline (7999)	Phase 3 (ours)	Δ
3-life clearance rate	95%	—	—
1-life clearance rate	0%	—	—
Ghosts eaten per episode	∼2–3	—	—
Perfect-clearance rate	0%	—	—

7 Related Work

Deep RL for Pac-Man. Early work on Berkeley Pac-Man competitions established evaluation-based function approximation (CE) as the dominant approach, with a reported ceiling of ∼13% win rate [1]. Our PPO baseline achieves 95% 3-life clearance, demonstrating the substantial advantage of modern on-policy methods and vectorized simulation over hand-crafted evaluation functions.

DecisionTransformer and Offline RL. Chen et al. [2] proposed conditioning on return-to-go for offline sequence modeling. Subsequent work [3] explored online fine-tuning. Our Observation 1 provides a mechanistic characterization of when and why this paradigm fails: highly-skewed return distributions cause the model's own noisy return estimates to serve as OOD-inducing perturbations at the conditioning input.

DAgger and Imitation Learning. Ross et al. [4] established the no-regret property of DAgger under stationary distributions. Our Observation 2 identifies a failure mode not covered by the original analysis: when the teacher has an expressive ceiling on the task, Bellman error minimization amplifies OOD annotation errors through the max operator, creating a self-reinforcing degradation pattern that additional data alone does not resolve.

World Models and Dreamer. Hafner et al. [8, 9] learn latent world models from pixels and plan via latent imagination. Our approach shares the intuition that forward models are valuable, but differs in implementation: rather than learning a world model, we directly encode the *known* deterministic forward dynamics of opponents as observation channels, eliminating the need to learn them from data.

HRA and Reward Decomposition. van Seijen et al. [14] introduced the Hybrid Reward Architecture, decomposing the value function into per-dimension heads corresponding to independent reward components. Juozapaitis et al. [10] extended this idea for explainable RL. Our experiments with HRA-style decomposition (Observation 4) are consistent with the interpretation that reward decomposition alone is insufficient when the observation space lacks the representational capacity to distinguish opponent behaviors—credit assignment and representation are complementary bottlenecks.

8 Conclusion

This paper reported on a comparative study of three deep RL architectures applied to Pac-Man. We observed two distinct behavioral patterns—OOD degradation in DecisionTransformer and distribution-aggregation degradation in DAgger+DQN—and described the properties of PPO+GAE that allowed it to avoid both. We further found that PPO's remaining weakness—zero 1-life clearance—appeared to stem from representational limitations in the observation space, and proposed an explicit spatial prior encoding as a way to address this. Two planned ablation studies aim to test whether the forward-prediction channels reduce the need for temporal frame stacking, and whether the learned policy transfers to unseen maze layouts.

In our Pac-Man experiments, **architectural choices—how data is collected, how credit is assigned, and what information is made explicit—appeared to matter more for learning stability than hyperparameter tuning or model capacity**. Whether this pattern holds in other tail-risk-dominated, zero-tolerance environments is an open empirical question.

References

[1] Berkeley Pac-Man Contest. *UC Berkeley CS188*. http://ai.berkeley.edu/contest

[2] L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, I. Mordatch. "Decision Transformer: Reinforcement Learning via Sequence Modeling." *NeurIPS*, 2021.

[3] Q. Zheng, A. Zhang, A. Grover. "Online Decision Transformer." *ICML*, 2022.

[4] S. Ross, G. J. Gordon, D. Bagnell. "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning." *AISTATS*, 2011.

[5] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov. "Proximal Policy Optimization Algorithms." *arXiv:1707.06347*, 2017.

[6] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, P. Abbeel. "High-Dimensional Continuous Control Using Generalized Advantage Estimation." *ICLR*, 2016.

[7] Y. Burda, H. Edwards, A. Storkey, O. Klimov. "Exploration by Random Network Distillation." *ICLR*, 2019.

[8] D. Hafner, T. Lillicrap, J. Ba, M. Norouzi. "Dream to Control: Learning Behaviors by Latent Imagination." *ICLR*, 2020.

[9] D. Hafner, J. Pasukonis, J. Ba, T. Lillicrap. "Mastering Diverse Domains through World Models." *arXiv:2301.04104*, 2023.

[10] Z. Juozapaitis, A. Koul, A. Ferns, M. Erwig, F. Doshi-Velez. "Explainable Reinforcement Learning via Reward Decomposition." *IJCAI*, 2019.

[11] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grew, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, D. Hassabis. "Mastering the Game of Go with Deep Neural Networks and Tree Search." *Nature*, 2016.

[12] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, D. Hassabis. "Human-Level Control through Deep Reinforcement Learning." *Nature*, 2015.

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin. "Attention Is All You Need." *NeurIPS*, 2017.

[14] H. van Seijen, M. Fatemi, J. Romoff, R. Laroche, T. Barnes, J. Tsang. "Hybrid Reward Architecture for Reinforcement Learning." *NeurIPS*, 2017.